

Retrieval Augmented Generation (RAG)

1. What is RAG?

RAG (Retrieval-Augmented Generation) is a method where an LLM doesn't answer only from its training memory.

Instead, it **retrieves relevant company documents** (PDF, FAQ, policies, DB) and then generates an answer using those references.

This makes responses **factual, transparent, and traceable**, which is required for enterprise AI systems.

2. Why RAG instead of Fine-Tuning?

Fine-tuning updates model **behavior + tone**, but doesn't update **knowledge**. If policy or product data changes, you must retrain again → **time, GPU cost, and maintenance burden**.

RAG injects **real-time updated knowledge** during inference, making it **cost-efficient, faster to deploy, and reliable** for live business use cases.

3. Explain RAG workflow.

RAG works step-by-step:

- Load docs (PDF/website/database)
- Clean & Chunk
- Create Embeddings
- Store in Vector DB
- User Query → Convert to embedding
- Retrieve Top-K matching chunks
- LLM answers using retrieved context

It connects **search + reasoning**, ensuring the model answers only from verified data.

4. What are Embeddings & why important?

Embeddings turn text into **semantic numerical vectors**.

They capture **meaning** instead of exact words.

Example: "leave rules" $\approx$ "vacation policy".

This enables the system to **understand context, synonyms, and concepts**, leading to smarter search.

5. Why use a Vector Database?

A Vector DB (FAISS / Pinecone / Chroma) efficiently stores & retrieves **millions of embeddings**.

It supports **fast similarity search, indexing, metadata filtering, and scalable retrieval**, essential for production-level RAG.

6. Why do we Chunk documents?

LLMs can't process huge files at once.

So we break docs into **logical chunks** (e.g., 300–600 tokens) to ensure:

- Better context matching
- Lower token usage
- More accurate retrieval

Correct chunking = **better answers**

7. Why document cleaning matters?

Documents include **headers, tables, numbers, noise, OCR errors**.

Cleaning ensures embeddings capture **real meaning**, not unwanted junk.

Clean text → **high-quality retrieval + less hallucination**.

8. How do you choose embedding model?

Consider:

- Domain fit (general / legal / medical)
- Performance accuracy
- CPU-friendly execution
- Multilingual support
- Latency & cost

Models like **BGE, MPNet, Sentence-T5** perform best in enterprise search.

9. What causes RAG latency & how optimize?

Latency causes:

- Slow vector search
- Heavy models
- Too many chunks retrieved
- Network overhead

Optimize via **HNSW index, batching, caching, tuned Top-K, lightweight embeddings**, and efficient deployment.

10. What is Hybrid Search & why needed?

Combines **keyword BM25 + semantic embeddings**.

Keyword ensures exact terms (ex: GST rate).

Semantic ensures meaning (ex: "holiday policy vs leave rules").

Hybrid gives **maximum recall + precision** — best for enterprise data.

11. Walk through complete RAG architecture.

A robust RAG system includes:

- Ingestion layer → read docs
- Pre-processing
- Chunking
- Embedding service
- Vector DB
- Retrieval logic
- LLM reasoning
- Formatting + Output templates

This ensures **accuracy, speed, and structured responses.**

12. What is hallucination & how reduce?

Hallucination = model gives **confident but wrong answers.**

Reduce via:

- Grounding prompt
- RAG retrieval
- Citations
- Verification loops
- Hybrid search

RAG anchors **answers to real evidence.**

13. What is a grounding prompt?

A prompt instructing LLM to answer only from **given context.**

Example:

"If info not in provided text, respond 'Not found in context'."

This enforces truth-based answering.

14. Fine-tuning vs RAG – when choose?

Use Fine-tuning for:

- Tone/style personalization
- Domain language learning (legal phrasing)

Use RAG for:

- Changing knowledge
- Document-based answering
- Low-cost high-accuracy use

RAG » FT for dynamic business knowledge.

15. How do you evaluate RAG quality?

Use:

- RAGAS metrics (faithfulness, relevance)
- Manual testing
- User feedback logs
- Reference vs answer comparison

Goal = **accuracy + trust + business usefulness.**

16. What is Top-K?

K = number of retrieved chunks.

Good range: 3–5.

Too high → noise

Too low → missing info

Proper K **improves precision & context coverage.**

17. How to handle noisy PDFs?

Approach:

- OCR
- Remove headers/footers/page numbers
- Fix broken text
- Table extraction
- Semantic chunking

Clean docs = **clean vectors = clear answers.**

18. Why prompt engineering crucial in RAG?

Good prompts teach model to:

- Use retrieved context
- Follow format
- Cite sources
- Avoid assumptions

Prompt = **model behavior control layer.**

19. What is context window?

Max tokens model can read at once.

Bigger window = **more document context** → fewer misses.

Helps for long **PDF / legal docs / research RAG.**

20. Why chunk overlap?

Overlap keeps **continuity** between chunks.

Prevents cutting mid-sentence.

Improves **recall & retrieval quality.**

21. Semantic vs Keyword search?

- **Keyword** = exact word match (BM25)
- **Semantic** = meaning match (embeddings)

Hybrid = **best relevance for real business docs.**

22) Retrieval latency? How reduce?

Reduce through:

- Index tuning (HNSW)
- GPU/CPU choice
- Query caching
- Chunk size optimization
- Top-K tuning

Goal = fast, chat-like response.

23.What is Self-RAG?

Model **retrieves, verifies, corrects by itself.**

Good for **autonomous agents**, safety-critical use cases (health, finance, legal).

24.Why use Agents in RAG?

Agents add layers like:

- Planning
- Retrieval
- Cross-checking
- Summarization
- Citation validation

Agents = **Reliable, auditable AI pipelines.**

25.Data freshness issue — how RAG solves?

LLM static memory = outdated.

RAG injects **latest docs instantly**, ensuring always-current knowledge.

Useful for **policies, medical updates, compliance, product docs**.

26.What is Retrieval Noise?

Noise = **irrelevant chunks**.

Fix via:

- Better chunk strategy
- Hybrid search
- Re-rankers
- Metadata filtering

Less noise → cleaner LLM context.

27.What is Vector Drift?

Changing embedding model shifts meaning space.

Never mix vectors.

Solution: **Re-embed entire store when model changes.**

28.How to store vectors reliably?

Use **persistent vector DB** (Pinecone/Chroma-persistent/Weaviate).

Avoid in-memory for production (data loss risk).

29. Why structured output needed?

For **API consumption, UI display, automation.**

JSON / bullet / key points prevent **messy free text** and help systems post-process efficiently.

30. Why hybrid RAG architecture preferred?

Hybrid RAG combines:

- Retrieval
- Re-ranking
- Grounding
- Agents
- Verification

This ensures **high accuracy, explainability, and trust**, required for enterprise deployments.

Scenario Based Questions

31. You're building a RAG chatbot for an HR policy PDF. The document has clear section headers like *Leave Rules* and *Travel Allowance*. How will you decide the chunking strategy?

I'll use **semantic chunking** based on **headings and section meaning** instead of fixed splits.

Each **chunk = 350–550 tokens with 15% overlap**, ensuring that related rules and exceptions stay together for **accurate retrieval**

32. Your SOP document (80 pages) produces vague answers after chunking with 2000 tokens. What's going wrong and how do you fix it?

The **chunks are too large**, pulling irrelevant policy data into the retrieval. I **reduce chunk size to 400–600 tokens, re-embed the corpus**, and get **crisper answers** with **lower cost**.

33. In a legal RAG chatbot, chunks are only 150 tokens, and answers sound broken. What's the issue?

Legal clauses are getting split mid-way, breaking semantic flow.

I switch to **semantic clause-based chunking** and increase chunk size to around **500 tokens** for **complete contextual understanding**

34. Your RAG bot gives correct but incomplete answers. What could be the reason?

Increase **Top-K** (e.g. 3→5)

Adjust **chunk size** & **overlap**

Use better **embedding/reranker**

Evaluate with **RAGAS** for **completeness** & **faithfulness**

35. For a medical chatbot, the query “What is the HbA1c threshold for diabetes?” gives partial results. What’s your retrieval fix?

I use **hybrid search** — combining lexical (**BM25**) for exact medical terms like **HbA1c** and **semantic embeddings** for meaning-based understanding. This gives the **highest precision**.

36. A finance RAG bot keeps misunderstanding the reimbursement policy. Chunking seems fine — what’s next?

The issue is **weak retrieval; BM25** was disabled. I switch to **hybrid search (BM25 + embeddings)** and add a **reranker**, which improves **accuracy** and **context alignment**.

37. Your RAG system retrieves general policies instead of rule exceptions. How do you improve relevance?

I integrate a **cross-encoder reranker**, which reorders retrieved passages based on **semantic similarity**, surfacing **exception-specific** results.

38. Your RAG bot creates a fake policy clause about leave rules. How will you reduce hallucination?

I enforce:

- Answer only from context
- Citations required
- Reject answers when context is missing
- This reduces **hallucination drastically**.

39. When the query is “Does our company provide pet leave?” and the docs have no such info, what should the bot reply?

The bot should respond:

“This information is not mentioned in the provided policy documents.”

This ensures **truthful behavior** and **hallucination prevention**.

40. You’re handling Tamil + English HR policy docs. How do you pick the right embedding model?

I select a multilingual embedding model such as **LaBSE** or **Cohere Multilingual**, ensuring proper **Tamil semantic understanding** and **cross-language retrieval**.

41. After upgrading your embedding model, searches return nonsense results. What’s the fix?

Old and new **vector representations** are inconsistent.

I **re-embed the complete corpus** and **rebuild the vector index** to eliminate **vector drift**.

42. Your RAG bot fails to answer correctly from reimbursement tables in a finance PDF. What’s the right preprocessing approach?

I extract tables using **Camelot**, **convert to structured rows**, and **embed** them separately.

This provides **accurate, table-specific answers**.

43. In HR chatbot, the query “maximum LTA per year?” gives partial results. How do you improve it?

I use both **lexical search** (to match “LTA max claim”) and **semantic search** (to understand policy meaning).

Combining both ensures **precise and context-aware retrieval**.

44. A query about travel policy retrieves medical policy content. How do you fix context leakage?

I apply **metadata filtering (doc_type = travel_policy)** and fine-tune **Top-K** for better relevance.

Now retrieval is **clean and domain-specific**.

45. Your RAG chatbot mixes 2021 and 2024 HR policies in responses. What's the solution?

I filter documents using **metadata fields** like year and doc_version.

This ensures only the **latest policies** are retrieved and used for answers.

46. You're choosing between small and large embedding models for RAG. Which do you pick and why?

If the **dataset is small and domain-specific**, I pick a **light embedding model** for **speed and low cost**.

For complex or multi-domain documents, I use a **larger model** (e.g., text-embedding-3-large) for **better semantic understanding**.

47. You need to deploy RAG at scale. How do you choose between FAISS, Pinecone, and Chroma?

- **FAISS**: Best for **local or offline** testing
- **Pinecone**: **Managed cloud** option, scalable & fast
- **Chroma**: **Lightweight, open-source** for small projects

Choice depends on **scale, latency, and budget**.

48. User asks a query spanning multiple documents. How will your RAG handle it?

I store document metadata (like doc_id) and allow **cross-document retrieval**.

Then I use a **reranker** to merge and reorder chunks for **multi-source context synthesis**.

49. Your bot fails when users type short or vague queries like “leave rules?”. What’s your fix?

I apply **query rewriting** or **expansion** using a **small LLM** to convert vague queries into **context-rich search prompts**, improving **retrieval accuracy**.

50. User queries in Tamil but documents are in English. How to maintain accuracy?

Use **multilingual embeddings** or **translate query → English → search → Tamil response**.

This ensures **semantic alignment** between user language and document language.

51. How do you evaluate your RAG system’s performance beyond accuracy?

I use **RAG metrics** —

- **Faithfulness** (context-based correctness)
- **Context Precision** (relevant retrieval)
- **Answer Relevance** (LLM match quality).

These metrics show **end-to-end system reliability**.

52. Embedding large documents is costly. What's your cost-saving strategy?

I perform **text deduplication, metadata filtering**, and **incremental embedding** (embed only new or changed data).

This reduces **embedding cost and time**.

53. Your retrieved results are semantically correct but poorly ranked. What do you do?

Add a **cross-encoder reranker** (e.g., **MS MARCO / MiniLM**) to **re-score** results after vector retrieval for **improved relevance**.

54. Sometimes your RAG model reveals internal reasoning like "I think the answer is...". How do you prevent this?

I use **strict prompt templates** that enforce **"Answer only from the given context"**.

Also, I add **system instructions** to hide reasoning traces and ensure **professional, factual responses**.

55. Your RAG chatbot answers correctly at first but slowly starts giving off-topic responses after document updates. What could be happening?

This is **semantic drift** — caused by **new embeddings replacing old context** inconsistently.

I fix it by **re-embedding the full dataset**, clearing **stale vectors**, and ensuring **consistent embedding models** across versions.

56. A user asks “Who approves medical leave?” but retrieval brings “leave types.” Why?

The **embedding model captured theme similarity** but not **intent-level meaning**.

I apply a **query expansion step** that reformulates “who approves” into **“approval process for medical leave”** before retrieval.

57. User inputs a 5-sentence paragraph query. How do you ensure proper retrieval?

I use **query summarization** — compress the long query into **key semantic intent** using a **mini-LLM** before retrieval.

This maintains **relevance and speed**.

58. You trained RAG for HR docs but now using it for medical policies — results drop. Why?

Embeddings are **domain-sensitive**. I retrain or fine-tune embeddings using **domain-specific corpora**, ensuring **vocabulary alignment** and **semantic relevance** in retrieval.

59. How do you ensure the final RAG answer matches the retrieved context?

Add a **validation layer** — use a **secondary LLM check** that verifies if the generated answer is **fully supported by the context**.

If not, it triggers a **fallback “insufficient data”** response.

60. You’re deploying RAG for internal company use. How do you protect confidential data?

I apply **access control** at the **retrieval level**, encrypt **vector data**, and use **role-based query filters**.

This ensures **data privacy** and prevents **information leakage** from restricted documents.

RAG Interview Questions

- 1.What is RAG?
- 2.Why RAG instead of Fine-Tuning?
- 3.Explain RAG workflow.
- 4.What are Embeddings & why important?
- 5.Why use a Vector Database?
- 6.Why do we Chunk documents?
- 7.Why document cleaning matters?
- 8.How do you choose embedding model?
- 9.What causes RAG latency & how optimize?
- 10.What is Hybrid Search & why needed?
- 11.Walk through complete RAG architecture.
- 12.What is hallucination & how reduce?
- 13.What is a grounding prompt?
- 14.Fine-tuning vs RAG — when choose?
- 15.How do you evaluate RAG quality?
- 16.What is Top-K?
- 17.How to handle noisy PDFs?
- 18.Why prompt engineering crucial in RAG?
- 19.What is context window?
- 20.Why chunk overlap?
- 21.Semantic vs Keyword search?
- 22.Retrieval latency? How reduce?
- 23.What is Self-RAG?
- 24.Why use Agents in RAG?
- 25.Data freshness issue — how RAG solves?
- 26.What is Retrieval Noise?
- 27.What is Vector Drift?
- 28.How to store vectors reliably?
- 29.Why structured output needed?
- 30.Why hybrid RAG architecture preferred?

31. You're building a RAG chatbot for an HR policy PDF. The document has clear section headers like Leave Rules and Travel Allowance. How will you decide the chunking strategy?
32. Your SOP document (80 pages) produces vague answers after chunking with 2000 tokens. What's going wrong and how do you fix it?
33. In a legal RAG chatbot, chunks are only 150 tokens, and answers sound broken. What's the issue?
34. Your RAG bot gives correct but incomplete answers. What could be the reason?
35. For a medical chatbot, the query "What is the HbA1c threshold for diabetes?" gives partial results. What's your retrieval fix?
36. A finance RAG bot keeps misunderstanding the reimbursement policy. Chunking seems fine – what's next?
37. Your RAG system retrieves general policies instead of rule exceptions. How do you improve relevance?
38. Your RAG bot creates a fake policy clause about leave rules. How will you reduce hallucination?
39. When the query is "Does our company provide pet leave?" and the docs have no such info, what should the bot reply?
40. You're handling Tamil + English HR policy docs. How do you pick the right embedding model?
41. After upgrading your embedding model, searches return nonsense results. What's the fix?
42. Your RAG bot fails to answer correctly from reimbursement tables in a finance PDF. What's the right preprocessing approach?
43. In HR chatbot, the query "maximum LTA per year?" gives partial results. How do you improve it?
44. A query about travel policy retrieves medical policy content. How do you fix context leakage?
45. Your RAG chatbot mixes 2021 and 2024 HR policies in responses. What's the solution?
46. You're choosing between small and large embedding models for RAG. Which do you pick and why?

47. You need to deploy RAG at scale. How do you choose between FAISS, Pinecone, and Chroma?
48. User asks a query spanning multiple documents. How will your RAG handle it?
49. Your bot fails when users type short or vague queries like “leave rules?”. What’s your fix?
50. User queries in Tamil but documents are in English. How to maintain accuracy?
51. How do you evaluate your RAG system’s performance beyond accuracy?
52. Embedding large documents is costly. What’s your cost-saving strategy?
53. Your retrieved results are semantically correct but poorly ranked. What do you do?
54. Sometimes your RAG model reveals internal reasoning like “I think the answer is...”. How do you prevent this?
55. Your RAG chatbot answers correctly at first but slowly starts giving off-topic responses after document updates. What could be happening?
56. A user asks “Who approves medical leave?” but retrieval brings “leave types.” Why?
57. User inputs a 5-sentence paragraph query. How do you ensure proper retrieval?
58. You trained RAG for HR docs but now using it for medical policies — results drop. Why?
59. How do you ensure the final RAG answer matches the retrieved context?
60. You’re deploying RAG for internal company use. How do you protect confidential data?